P2953R0
Forbid defaulting operator=(X&&) &&

Published Proposal, 2023-08-10

Author:
Arthur O'Dwyer

Audience:
EWG

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Draft Revision:
4

Abstract

Current C++ permits explicitly-defaulted special members to differ from their implicitly-defaulted counterparts in various ways, including parameter type and ref-qualification. This permits implausible signatures like `A& operator=(const A&) && = default`, where the left-hand operand is rvalue-ref-qualified. We propose to forbid such signatures.

Table of Contents

1 Changelog

2 Motivation and proposal

2.1 Interaction with P2952

2.2 "Deleted" versus "ill-formed"

2.3 Existing corner cases

2.4 Impact on existing code

3 Implementation experience

4 Proposed wording

4.1 [dcl.fct.def.default]

4.2 [class.copy.assign]

5 Proposed straw polls

References

Informative References

§ 1. Changelog

- R0:
  - Initial revision.

§ 2. Motivation and proposal

Currently, [dcl.fct.def.default]/2.5 permits an explicitly defaulted special member function to differ from the implicit one by adding *ref-qualifiers*, but not *cv-qualifiers*.

For example, the signature `const A& operator=(const A&) const& = default` is forbidden because it is additionally const-qualified, and also because its return type differs from the implicitly-defaulted `A&`. This might be considered unfortunate, because that's a reasonable signature for a const-assignable proxy-reference type. But programmers aren't clamoring for that signature to be supported, so we do not propose to support it here.

Our concern is that the *unrealistic* signature `A& operator=(const A&) && = default` is *permitted!* This has three minor drawbacks:

- The possibility of these unrealistic signatures makes C++ harder to understand. Before writing [P2952], Arthur didn't know such signatures were possible.
- The wording to permit these signatures is at least a tiny bit more complicated than if they weren't permitted.
- The quirky interaction with [CWG2586] and [P2952] discussed in the next subsection.

To eliminate all three drawbacks, we propose that a defaulted copy/move assignment operator should not be permitted to add to its implicit signature an *rvalue* ref-qualifier (nor an explicit object parameter of rvalue reference type).

§ 2.1. Interaction with P2952

[CWG2586] (adopted for C++23) permits `operator=` to have an explicit object parameter.

[P2952] proposes that `operator=` should (also) be allowed to have a placeholder return type. If P2952 is adopted without P2953, then we will have the following pub-quiz fodder:

```
struct C {
  auto&& operator=(this C&& self, const C&) { return self; }
    // Today: OK, deduces C&&
    // After P2952: Still OK, still deduces C&&
    // Proposed: Still OK, still deduces C&&

  auto&& operator=(this C&& self, const C&) = default;
    // Today: Ill-formed, return type involves placeholder
    // After P2952: OK, deduces C&
    // Proposed: Deleted, object parameter is not C&
};
```

The first, non-defaulted, operator "does the natural thing" by returning its left-hand operand, and deduces `C&&`. The second operator also "does the natural thing" by being defaulted; but after P2952 it will deduce `C&`. (For rationale, see [P2952] §3.3 "Deducing `this` and CWG2586.") The two "natural" implementations deduce different types! This might be perceived as inconsistency.

If we adopt P2953 alongside P2952, then the second `operator=` will go back to being unusable, which reduces the perception of inconsistency.

	Today	**P2952**
Today	`C&&`/ill-formed	`C&&`/`C&`
P2953	`C&&`/ill-formed	`C&&`/deleted

§ 2.2. "Deleted" versus "ill-formed"

[dcl.fct.def.default] goes out of its way to make many explicitly defaulted assignment operators "defaulted as deleted," rather than ill-formed. I think I understand the reason for this in the case of *comparison operators* (see [P2952] §3.2

"Defaulted as deleted"), but it's non-obvious why we should care about the corresponding cases for constructors, destructors, and assignment operators. (Clang handles this example correctly; GCC, MSVC, and EDG already non-conformingly treat *both* `cl` and `cr` as ill-formed.)

```
template<template<class> class TT>
struct C {
  C& operator=(TT<const C>) = default;
};

C<std::add_lvalue_reference_t> cl;
  // OK, operator= is defaulted
  // (and GCC/MSVC/EDG have a bug)

C<std::add_rvalue_reference_t> cr;
  // OK, operator= is defaulted as deleted
  // (but why not just make it ill-formed?)
```

P2953 isn't yet proposing to change the complicated status quo, but Arthur would certainly like to learn the status quo's rationale. If we were willing to aggressively change the status quo, we could simplify [dcl.fct.def.default] something like this:

> 1. A function definition whose *function-body* is of the form `= default ;` is called an *explicitly-defaulted* definition. A function that is explicitly defaulted shall
>
> - (1.1) be a special member function or a comparison operator function ([over.binary]), and
> - (1.2) not have default arguments.
>
> 2. An explicitly defaulted special member function F_1 is allowed to differ from the corresponding special member function F_2 that would have been implicitly declared, as follows:
>
> - (2.1) if F_2 is an assignment operator, ~~F_1 and F_2 may have differing *ref-qualifiers*~~ F_1 may have an lvalue *ref-qualifier*;
> - (2.2) if F_2 ~~has an implicit object parameter of type "reference to C"~~ is an assignment operator with an implicit object parameter of type `C&`, F_1 may ~~be an explicit object member function whose~~ have an explicit object parameter ~~is~~ of type ~~(possibly different) "reference to C"~~ `C&`, in which case the type of F_1 would differ from the type of F_2 in that the type of F_1 has an additional parameter; and
> - (2.3) F_1 and F_2 may have differing exception specifications~~.~~ ~~; and~~
> - ~~(2.4) if F_2 has a non-object parameter of type `const C&`, the corresponding non-object parameter of F_1 may be of type `C&`.~~
>
> If the type of F_1 differs from the type of F_2 in a way other than as allowed by the preceding rules, then:
>
> - ~~(2.5) if F_1 is an assignment operator, and the return type of F_1 differs from the return type of F_2 or F_1's non-object parameter type is not a reference, the program is ill-formed;~~
> - (2.6) ~~otherwise,~~ if F_1 is a three-way comparison operator explicitly defaulted on its first declaration, it is defined as deleted;
> - (2.7) otherwise, the program is ill-formed.
>
> [...]

Again, P2953 doesn't yet propose the above change; but it would be good to know why we shouldn't.

§ 2.3. Existing corner cases

There is vendor divergence in some corner cases. Here is a table of the divergences we found, plus our opinion as to the conforming behavior, and our proposed behavior.

URL	Code	Clang	GCC	MSVC	EDG
link	`C& operator=(C&) = default;`	✓	✓	✓	✓
link	`C& operator=(const C&&) = default;`	deleted	✗	✗	deleted
link	`C& operator=(const C&) const = default;`	deleted	✗	✗	deleted
link	`C& operator=(const C&) && = default;`	✓	✓	✓	✓
link	`C&& operator=(const C&) && = default;`	✗	✗	✗	✗

link	`template<class>`
	`struct C {`
	`  C& operator=(std::add_lvalue_reference_t<const C>) = default;`
	`};`
		✓	✗	✗	✗

§ 2.4. Impact on existing code

This proposal takes code that was formerly well-formed C++23, and makes it ill-formed. The affected constructs are extremely implausible in Arthur's opinion; but of course we need some implementation and usage experience in a real compiler before adopting this proposal.

```
struct C {
  C& operator=(const C&) && = default;
    // Today: Well-formed
    // Tomorrow: Deleted
};

struct D {
  D& operator=(this D&& self, const C&) = default;
    // Today: Well-formed
    // Tomorrow: Deleted
};
```

§ 3. Implementation experience

None yet.

§ 4. Proposed wording

§ 4.1. [dcl.fct.def.default]

> NOTE: The only defaultable special member functions are default constructors, copy/move constructors, copy/move assignment operators, and destructors. Of these, only the assignment operators can ever be cvref-qualified at all.

Modify [dcl.fct.def.default] as follows:

1. A function definition whose *function-body* is of the form `= default ;` is called an *explicitly-defaulted* definition. A function that is explicitly defaulted shall

- (1.1) be a special member function or a comparison operator function ([over.binary]), and
- (1.2) not have default arguments.

2. An explicitly defaulted special member function F_1 is allowed to differ from the corresponding special member function F_2 that would have been implicitly declared, as follows:

- (2.1) if F_2 is an assignment operator, ~~F_1 and F_2 may have differing *ref-qualifiers*~~ F_1 may have an lvalue *ref-qualifier*;
- (2.2) if F_2 ~~has an implicit object parameter of type "reference to C"~~ is an assignment operator with an implicit object parameter of type `C&`, F_1 may ~~be an explicit object member function whose~~ have an explicit object parameter ~~is~~ of type ~~(possibly different) "reference to C"~~ `C&`, in which case the type of F_1 would differ from the type of F_2 in that the type of F_1 has an additional parameter;
- (2.3) F_1 and F_2 may have differing exception specifications; and
- (2.4) if F_2 has a non-object parameter of type `const C&`, the corresponding non-object parameter of F_1 may be of type `C&`.

If the type of F_1 differs from the type of F_2 in a way other than as allowed by the preceding rules, then:

- (2.5) if ~~F_1~~ F_2 is an assignment operator, and the return type of F_1 differs from the return type of F_2 or F_1's non-object parameter type is not a reference, the program is ill-formed;
- (2.6) otherwise, if F_1 is explicitly defaulted on its first declaration, it is defined as deleted;
- (2.7) otherwise, the program is ill-formed.

[...]

§ 4.2. [class.copy.assign]

NOTE: If we do the wording patch above, then I think nothing in [class.copy.assign] needs to change. But much of the wording above is concerned specifically with copy/move assignment operators, so it might be nice to move that wording out of [dcl.fct.def.default] and into [class.copy.assign]. Also note that right now a difference in `noexcept`-ness is handled explicitly by [dcl.fct.def.default] for special member functions but only by omission-and-note in [class.compare] for comparison operators.

Modify [class.copy.assign] as follows:

TODO FIXME BUG HACK

§ 5. Proposed straw polls

The next revision of this paper (if any) will be guided by the outcomes of these two straw polls.

	SF	***F***	***N***	***A***	***SA***
EWG would like to forbid rvalue-ref-qualified assignment operators (by any means, not necessarily by this proposed wording).	–	—	—	—	—
P2953R1 should pursue §2.2's wording, making some "defaulted-as-deleted" operators into hard errors.	–	—	—	—	—

§ References

§ Informative References

[CWG2586]

Barry Revzin. *Explicit object parameter for assignment and comparison*. May–July 2022. URL: https://cplusplus.github.io/CWG/issues/2586.html

[P2952]

Arthur O'Dwyer; Matthew Taylor. *auto& operator=(X&&) = default*. August 2023. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2952r0.html